Phase 3.1 — Meeting 4A

Digital Harmonium Workstation

Firmware State Machine Architecture (FSM Blueprint)

Status: Final Design Draft
Purpose: Define the complete high-level firmware state machine before implementing runtime states.

---

1. Objective

The Digital Harmonium Workstation is fundamentally a real-time embedded system.

Although the musician experiences it as a musical instrument, internally the ESP32 behaves like a deterministic operating system.

At every instant:

- the system is in exactly one **System State**
- each active voice is in exactly one **Voice State**
- each background subsystem has its own independent internal state

Before implementing any firmware, these states must be designed.

This document defines the complete firmware navigation architecture.

A later document ("Runtime State Architecture") will define every individual runtime state in detail.

---

2. Philosophy

The firmware must never become a collection of random functions.

Instead:

Everything the ESP32 does must belong to an explicit state.

Every state has:

- entry conditions
- responsibilities
- valid transitions
- exit conditions

No hidden behavior should exist.

This philosophy makes debugging dramatically easier.

---

3. Layers of State Machines

The firmware is divided into multiple independent state machines.

```
System State Machine
        │
        ├──────── UI State Machine
        │
        ├──────── Audio Engine
        │
        ├──────── Voice State Machines
        │
        ├──────── Recording Engine
        │
        ├──────── Playback Engine
        │
        ├──────── Storage Manager
        │
        └──────── Background Services
```

Each layer has a different responsibility.

---

4. Top-Level System State Machine

Only one System State may exist at any instant.

```
Power Off
    │
    ▼
Boot
```

```
    │
    ▼
Self Test
    │
    ▼
Initialization
    │
    ▼
Home Screen
    │
    ├──────── Voice Menu
    ├──────── Tabla Menu
    ├──────── Tanpura Menu
    ├──────── Recording Menu
    ├──────── Playback Menu
    ├──────── Settings Menu
    ├──────── Instrument Loading
    ├──────── USB Mode
    ├──────── Error Recovery
    └──────── Shutdown
```

This is the master controller of the firmware.

No subsystem may operate outside a valid System State.

---

5. Boot Sequence

Power ON always follows exactly the same order.

```
Power Applied
      │
      ▼
CPU Reset
      │
      ▼
Bootloader
      │
      ▼
Firmware Starts
      │
      ▼
Hardware Initialization
      │
```

```
      ▼
Self Test
      │
      ▼
System Initialization
      │
      ▼
Load Default Configuration
      │
      ▼
Enter Home Screen
```

This sequence is deterministic.

---

6. Self-Test Phase

During startup, the workstation performs a quick health check.

Items tested include:

- ESP32 initialized
- Display operational
- Audio amplifier responds
- DAC/I2S interface initialized
- SD card detected (if present)
- Internal flash accessible
- LED matrix operational
- Button matrix operational
- Keyboard matrix operational
- Configuration successfully loaded

Optional:

- brief audio beep
- LED sweep animation

Total boot time target:

3–5 seconds.

---

7. Home State

The Home Screen is the central hub.

Responsibilities:

- display currently selected instrument
- display status indicators
- receive navigation inputs
- launch subsystem menus
- continue audio playback with minimal CPU overhead

No heavy processing occurs here.

---

8. Primary Navigation States

From Home, the musician may enter one subsystem at a time.

```
Home

├── Voice Menu
├── Tabla Menu
├── Tanpura Menu
├── Recording
├── Playback
├── Settings
├── Instrument Loading
└── Shutdown
```

These are mutually exclusive UI states.

---

9. Voice Menu

Purpose:

Configure the current instrument.

Possible options:

- Instrument selection
- Transpose
- Root key information
- Instrument metadata
- Future effects (reverb, chorus)

Changing instrument transitions into:

---

10. Tabla Menu

Purpose:

Control rhythmic accompaniment.

Possible parameters:

- Start
- Stop
- Taal selection
- Tempo
- Transpose
- Future swing options

Tabla playback continues after leaving the menu.

---

11. Tanpura Menu

Purpose:

Configure drone accompaniment.

Possible parameters:

- Start
- Stop
- Scale
- Tempo
- Transpose

Like Tabla, playback continues in the background.

---

12. Recording Menu

Purpose:

Manage recording sessions.

Possible actions:

- Begin recording
- Stop recording
- Save recording
- Delete recording

Recording does not interrupt instrument playback.

---

13. Playback Menu

Purpose:

Replay recorded performances.

Possible actions:

- Select recording
- Play
- Pause
- Stop

During playback:

- LEDs reproduce key presses
- accompaniment remains synchronized when applicable

---

14. Settings Menu

Reserved for infrequently changed parameters.

Examples:

- LCD contrast
- Speaker volume
- Factory reset

- Diagnostic information
- Firmware version

Most musicians rarely visit this screen.

---

15. Instrument Loading State

Whenever a different instrument is selected:

```
Current Instrument
        │
        ▼
Unload Existing Instrument
        │
        ▼
Load VHL
        │
        ▼
Build Runtime Tables
        │
        ▼
Ready
```

This state blocks only instrument switching.

Other configuration remains intact.

---

16. USB Mode

Reserved for future expansion.

Possible uses:

- firmware update
- VHL transfer
- debugging
- MIDI over USB

Not required for initial implementation.

---

17. Error Recovery

If a recoverable fault occurs:

Examples:

- missing VHL
- SD card removed
- invalid recording

System enters Error State.

Responsibilities:

- display message
- stop unsafe operations
- preserve user configuration where possible
- return safely to Home

The firmware should never crash.

---

18. Shutdown State

Responsibilities:

- stop audio
- flush recordings
- close files
- save configuration
- disable peripherals

Safe shutdown prevents corruption.

---

19. Allowed System Transitions

```
Boot
  ↓
Initialization
  ↓
Home

Home
```

```
├── Voice Menu
├── Tabla Menu
├── Tanpura Menu
├── Recording
├── Playback
├── Settings
├── Loading
└── Shutdown

Any State
    │
    ▼
Error Recovery

Error Recovery
      │
      ▼
Home
```

No illegal transitions should exist.

---

20. Design Rules

The firmware follows these principles:

- Every operation belongs to one defined state.
- States should be small and focused.
- Long operations (loading instruments, saving recordings) receive dedicated states.
- Background services continue independently where appropriate.
- The musician should never be confused about the current operating mode.
- Navigation should require the minimum number of button presses.
- Future features must integrate by adding states rather than modifying existing ones.

---

21. Relationship to Future Documents

This document defines **where the firmware can be**.

It does **not** define:

- audio processing
- ADSR calculations
- polyphony allocation

- voice lifetime
- DSP mathematics
- recording data structures
- scheduler timing

These will be specified in later Phase 3 documents.

---

22. Next Step

The next architectural document is:

Runtime State Architecture

Unlike this document, it will define every operational state inside the firmware, including:

- detailed system runtime states
- state responsibilities
- entry conditions
- exit conditions
- transition rules
- implementation strategy

It will serve as the direct blueprint for the ESP32 firmware implementation.

End of Meeting 4A — Firmware State Machine Architecture